

Scientific discovery as meta-optimization: a combinatorial optimization case study

Yuan-Hang Zhang

yuz092@ucsd.edu

University of California San Diego <https://orcid.org/0000-0002-5415-3307>

Chesson Sipling

University of California San Diego

Massimiliano Di Ventra

University of California, San Diego <https://orcid.org/0000-0001-9416-189X>

Article

Keywords:

Posted Date: March 16th, 2026

DOI: <https://doi.org/10.21203/rs.3.rs-9108409/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: There is **NO** Competing Interest.

Scientific discovery as meta-optimization: a combinatorial optimization case study

Yuan-Hang Zhang,^{1,*} Chesson Sipling,^{1,†} and Massimiliano Di Ventra^{1,‡}

¹*Department of Physics, University of California San Diego, La Jolla, CA 92093*

Scientific discovery is fundamentally an optimization problem, defined by a vast “state space” of theories and experiments, and an evaluation criterion based on quality, novelty, and validity. Large language models (LLMs) have enabled automated exploration of this space, but we argue that simultaneous modification of the evaluation criteria is equally important. Here, we propose formalizing research as *meta-optimization*, where the optimization objective itself is also being optimized. Our key contribution is “consensus objective aggregation,” where LLM-generated objective functions are combined via correlation-weighted voting, yielding a stable, self-correcting evaluation criterion that evolves as understanding deepens. We apply this framework to algorithm discovery for 3-SAT problems based on digital MemComputing machines, reducing the baseline scaling exponent for problem size N from $N^{2.51}$ to $N^{1.33}$ and delivering a $\sim 67\times$ speedup on the largest instances tested. As a problem-agnostic framework, we hope this approach will considerably aid scientific discovery.

I. INTRODUCTION

At its core, scientific research is an optimization process over a vast space of theories, experiments, and implementations [1]. Every published result corresponds to a local optimum in this “research space,” found through a slow, noisy sampling process carried out by humans. Evaluating a single idea may take months, and mistakes are common. Recently, large language models (LLMs) have started to automate this research process at scales far exceeding individual researchers, covering the entire loop from generating hypotheses, writing code, designing experiments, to analyzing results and writing academic papers [2–6].

The early results are striking. The AI Scientist [2] and its successor [3] produce complete research papers across machine learning subfields. Google’s AI co-scientist [4] generates and debates biomedical hypotheses through a tournament-style loop. FunSearch [7], AlphaEvolue [8], and SATLUTION [9] pair LLMs with evolutionary search to push the state of the art on algorithm discovery. Multi-agent systems now tackle materials discovery [10, 11], chemical synthesis [5], and the recovery of physical laws [12]. Tree-search methods, including Monte Carlo Tree Search (MCTS), have been coupled with LLMs for heuristic design [13–15] and mathematical reasoning [16, 17]. For surveys of this rapidly expanding area, see Refs. [18–21].

A more fundamental question, though, runs beneath all of these efforts: What should these systems actually optimize? The objectives guiding automated discovery, ranging from hand-crafted benchmarks, proxy scores, to LLM-generated evaluation functions, are imperfect measures for genuine research progress. This is Goodhart’s law in action, well known from reinforcement learning: “when a measure becomes a target, it ceases to be a good

measure” [22]. Optimizing against proxy objectives invites reward hacking, in which solutions score well on the metric while failing at the underlying goal [23, 24]. Objective specification has been flagged as a central difficulty for AI-driven discovery [25], and recent work on evolving objectives [25, 26] has begun to address it—though usually by swapping one objective for the next in sequence.

In real scientific research, the goals are often unclear or evolving. A project may start with something like “develop an efficient algorithm” or “discover a potent drug,” but as understanding builds, the criteria for success shift. Goals are typically plural—accuracy, explanatory power, robustness, cost, novelty, societal impact—and may resist any single stable ordering. It is possible to define a meta-objective and optimize the objective function itself, but this simply pushes the same specification and gaming problem up one level. Therefore, we treat objectives as evolving, uncertain, and multi-dimensional, working with portfolios rather than individual metrics.

In this work, we then propose *consensus objective aggregation* as a robust mechanism for *meta-optimization* in automated scientific discovery. Instead of maintaining a single evolving objective, the framework generates many objective functions, each encoding a different aspect of solution quality, and aggregates them through correlation-weighted voting. We compute pairwise Kendall’s τ rank correlations [27] across all objectives, weight each one by its median agreement with the rest (clamped to be non-negative) times an exponential age decay factor, and produce a consensus ranking via a weighted Borda count [28].

Since each proxy objective is an imperfect attempt to capture the true, unknown research goal, objectives that agree with each other are more likely to capture genuine research progress, while disagreeing outliers are likely noisy or misleading. Therefore, we assign higher weight to objectives that agree with many others. The age decay mechanism allows newer objectives to gradually replace older ones informed by updated understanding. Noisy or adversarial objectives are automatically suppressed, making the consensus ranking self-correcting.

* yuz092@ucsd.edu

† csipling@ucsd.edu

‡ diventra@physics.ucsd.edu

This consensus mechanism sits inside a multi-agent system that iterates in a closed loop, where objective evolution is steered by a meta-agent and carried out by an objective agent, while actual research is outlined by the planner agent and implemented by the designer agent. We apply the framework to algorithm discovery for random planted 3-SAT problems [29] using digital memcomputing machines (DMMs) [30–33]. Across the search, the system explored 414 solver designs under the guidance of 42 co-evolving objectives. The best solver found cuts the baseline scaling exponent for problem size N from $N^{2.51}$ to $N^{1.33}$ and achieves a $\sim 67\times$ speedup on the largest test instance. Of course, this LLM-based meta-optimization can be applied to a wide variety of problems. It is then our hope that it will considerably aid future scientific discovery.

II. RESULTS

A. Framework Overview

Our framework has four LLM agents in an iterative cycle (Fig. 1). Starting from a high-level goal defined by the human researcher, each iteration proceeds through the following stages:

Meta-agent. The meta-agent receives the human-defined research goal and steers the overall research strategy. Periodically, it analyzes existing objectives via Kendall’s τ correlations, providing strategic guidance to the objective agent and assigning weight multipliers that can amplify useful objectives or suppress harmful ones.

Objective agent. Acting on the meta-agent’s assessment, the objective agent generates new proxy objective functions mapping experiment results to scalar quality scores. Each objective is meant to capture a different aspect of solution quality. All objectives are scored on all designs and aggregated into a single consensus ranking through Kendall’s τ correlation-weighted voting with age decay (Sec. IIB and Sec. IVB).

Planner agent. The planner receives the MCGS-ranked design list (scored by the consensus objective) along with the full experiment history. It looks for patterns among successes and failures and outputs several strategic research directions, with references for each direction to build upon.

Designer agent. For each direction the planner proposes, the designer writes new solver code and runs it through a multi-fidelity experiment schedule [34]. Designs advance to higher fidelity levels by rule-based criteria relative to the current population, so that computational budget concentrates on the strongest candidates. The heteroscedastic evolutionary bayesian optimization (HEBO) algorithm [35] periodically tunes hyperparameters on the best untuned design.

This iterative cycle allows solutions and evaluation criteria to evolve simultaneously. The consensus mechanism ensures a stable research objective: even if certain LLM-

generated objectives could be misleading, the consensus ranking stays robust and informative.

B. Consensus Objective Aggregation

No single proxy objective reliably captures solution quality. An objective rewarding low computational cost at small problem sizes, for example, may unintentionally favor solutions that scale poorly. Each time an LLM generates a new objective function, it can introduce subtle biases or overlook important edge cases. Depending on any one such objective, even one that evolves over time, can lead to reward hacking.

Instead, we maintain a portfolio of objective functions and aggregate them into a consensus ranking. The intuition is straightforward: every LLM-generated objective is an imperfect proxy for the same underlying latent research goal. Good proxies should more or less agree, because they each approximate the same target from different aspects; an objective that disagrees with the majority has likely drifted too far from that goal through subtle biases or blind spots. The aggregation procedure (Fig. 2) works as follows:

1. **Score matrix.** Each objective function f_i scores every design d_j , producing a matrix $S_{ij} = f_i(d_j)$.
2. **Rank conversion.** Scores are converted to ranks within each objective (lower score = better rank), keeping the score range consistent across different objectives.
3. **Pairwise correlations.** We compute the Kendall’s τ rank correlation [27] τ_{ik} between every pair of objectives f_i and f_k over all designs. The complete Kendall’s τ matrix is plotted in Fig. 2(a), revealing clusters of positively correlated objectives and a few outliers.
4. **Agreement weighting.** Each objective’s weight is proportional to its median pairwise τ with all other objectives (clamped at zero) multiplied by an exponential age decay: $w_i = \max(\tilde{\tau}_i, 0) \cdot \lambda^{t-t_i}$, where $\tilde{\tau}_i = \text{median}_{k \neq i}(\tau_{ik})$ is the median correlation, $\lambda = 0.9$ is the decay base, t is the current round, and t_i is the round in which objective i was created. Weights are then normalized to sum to one. The weights of all LLM-generated objectives are plotted in Fig. 2(b).
5. **Consensus ranking.** The final ranking is a weighted Borda count [28]: $C_j = \sum_i w_i \cdot R_{ij} / (n-1)$, where R_{ij} is the rank of design d_j under objective f_i and n is the number of designs.

This aggregation procedure is self-correcting. Objectives that disagree with the majority get near-zero weight through the median τ clamping, and age decay gradually

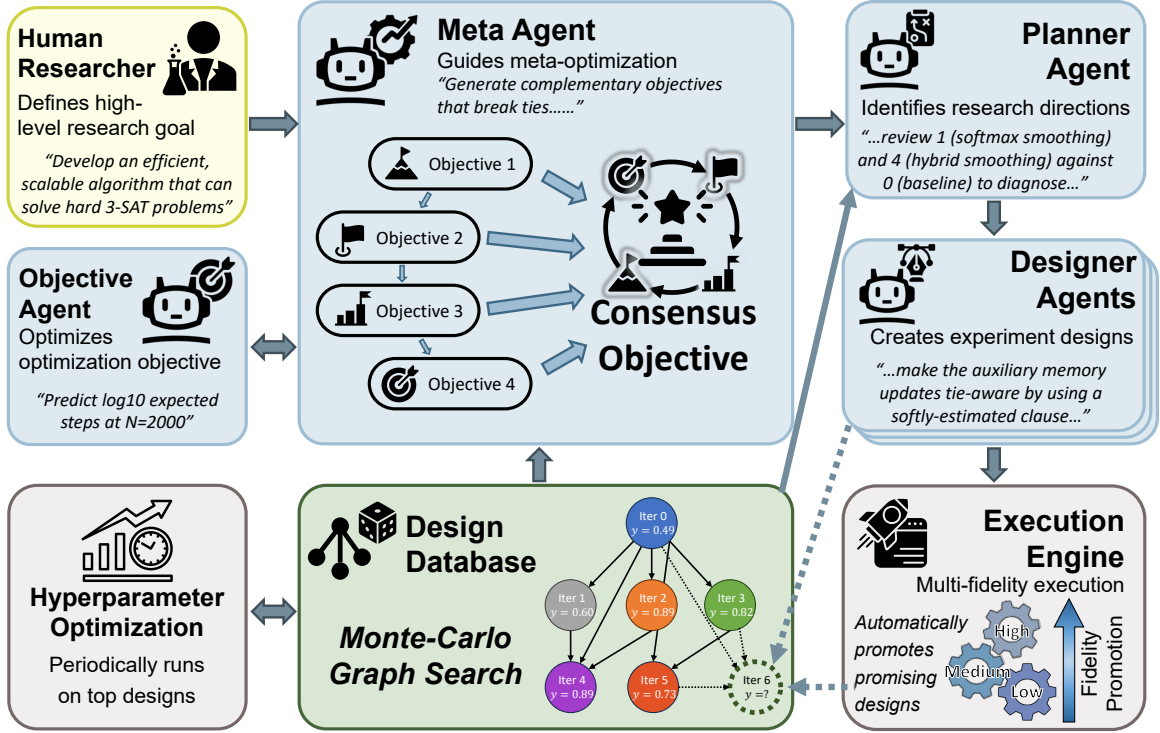


FIG. 1. **Framework overview.** The system consists of four LLM agents in an iterative cycle. Starting from a high-level human-designed research goal, the *meta-agent* sets the research strategy, guiding objective generation and analyzing objective quality. The *objective agent* proposes proxy objective functions reflecting different aspects of solution quality; these feed into a *consensus objective* that aggregates rankings through correlation-weighted voting. The *planner agent* uses Monte Carlo Graph Search (MCGS) under the consensus objective to identify strategic research directions. The *designer agent* turns those directions into concrete implementations, which are tested by a *multi-fidelity execution engine* that allocates computational budget to the most promising designs. A *hyperparameter optimization* module periodically tunes the leading design’s parameters.

shifts influence from older objectives to newer, better-informed ones.

Principal component analysis (PCA) on the standardized rank vectors of all objectives (Fig. 2(c)) confirms this picture: objectives that rank designs similarly cluster together, while outlier objectives sit apart and receive low consensus weight. The earlier objectives form an initial, smaller cluster, which gradually shifts towards another larger cluster as understanding deepens and the system’s notion of quality converges. The Supplementary Information (SI), Sec. II traces the full evolution of the objective portfolio, including reward hacking episodes and their mitigation.

Still, agreement-based weighting does have a blind spot. When a block of correlated objectives constitutes the majority, mutual agreement inflates every member’s weight, even if the criterion they collectively encode is misaligned with the true research goal. The consensus mechanism cannot tell a genuinely informative majority from an echo chamber of redundant proxies.

To address this, the meta-agent periodically reviews all objectives and assigns weight multipliers m_i that modulate the consensus weights ($w'_i \propto w_i \cdot m_i$). This adjustment can break cluster dominance where statistical

aggregation alone cannot. In the 3-SAT case study we discuss below, the meta-agent used this to progressively zero out 20 early objectives whose tight mutual correlation rewarded small- N performance, redirecting weight to newer metrics better matched to the large- N regime (see SI Sec. IIB for details).

C. Exploration via Monte Carlo Graph Search

The exploration-exploitation dilemma [36] is a well-studied problem and has standard solutions in bandit-style problems [37]. In our framework, we balance the exploration of novel designs and exploitation of promising designs via Monte Carlo Graph Search (MCGS) [38]. Each new design is built from one or more reference designs, and this parent-child structure forms a directed graph, which is the basis for MCGS.

We adapt the Upper Confidence Bound (UCB) algorithm [37, 39, 40] to our design graph. Each design node

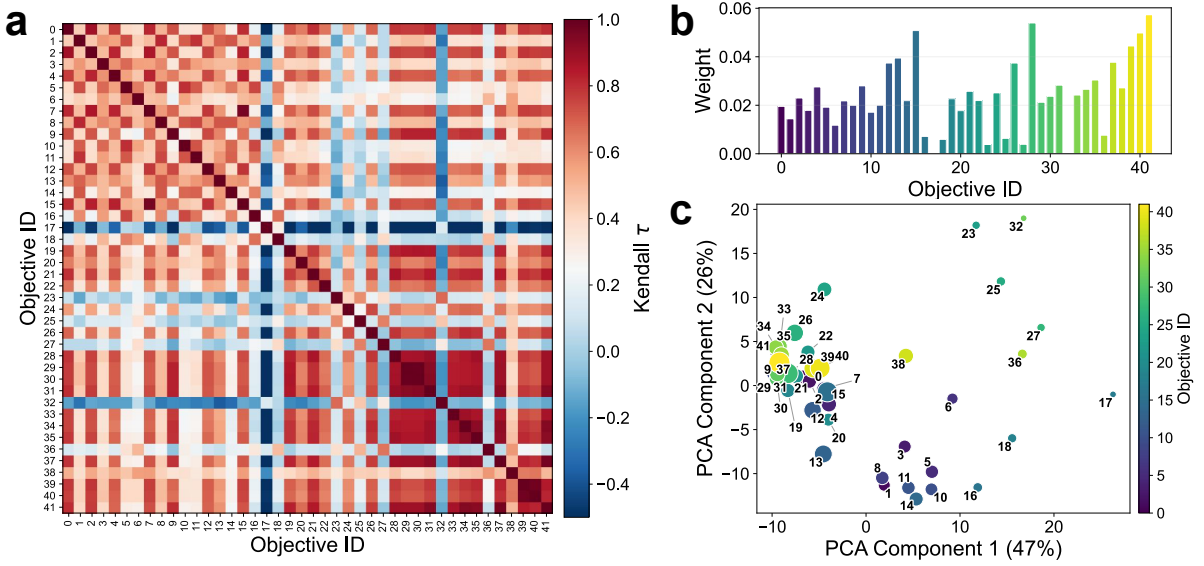


FIG. 2. **Consensus objective aggregation.** (a) Kendall’s τ correlation matrix across 42 LLM-generated objectives. Most objectives are positively correlated (red), but a few outliers have negative correlations with the majority (blue), showing that LLM-generated objectives can indeed be misleading sometimes. (b) Consensus weights after correlation-weighted voting with age decay ($\lambda = 0.9$). Newer objectives are weighted higher due to age decay, while some older objectives still carry a large weight from the correlated majority. Outlier objectives with low agreement are suppressed. (c) PCA of objectives based on pairwise τ correlations, colored by objective ID (creation order); larger dots denote higher weight. Earlier objectives form a small, isolated cluster, while later objectives converge towards another larger cluster, reflecting the shift in the research goal as research progresses. Outlier objectives are visually separated and get suppressed in the consensus ranking.

j is scored by

$$\text{UCB}_j = \underbrace{r_j}_{\text{exploitation}} + c \cdot \underbrace{\frac{\sqrt{N_{\text{total}}}}{1 + n_j}}_{\text{exploration}}, \quad (1)$$

where $r_j \in [0, 1]$ is the normalized rank score from the consensus objective (higher is better), n_j is the visit count for design j , N_{total} is the sum of all visit counts, and $c = 0.1$ is the exploration constant.

Visit counts measure how thoroughly a lineage has been explored. Our definition is different from standard MCTS [39, 40] to accommodate the graph structure, variable parent influences, and the fact that any node can be expanded. When a design is created, the designer agent self-reports how much each parent contributed (weights summing to 1). These weighted counts propagate upward through the genealogy by breadth-first search with a depth-increasing decay ($\kappa = 0.9$), concentrating credit near the immediate parents. Designs referenced often as parents accumulate high visit counts, shrinking their exploration bonus and moving the planner toward less-explored but potentially rewarding parts of the design space.

As the graph is rebuilt every iteration from the current consensus objective, rankings always reflect the latest evaluation criteria. The planner agent picks high-UCB designs as starting points for new research directions.

D. Application: Algorithm Discovery for Combinatorial Optimization

We test our framework on a challenging problem: designing efficient solvers for planted random 3-SAT instances (instances from Ref. [29]) near the complexity peak. The baseline digital MemComputing machine (DMM) solver [30, 32] is a physics-inspired dynamical system, whose fixed points encode satisfying assignments of the Boolean formula (see Sec. IV E). A key challenge is that solver performance at certain sizes does not necessarily predict scaling: a solver that looks fast at $N \lesssim 100$ variables may scale poorly to $N > 1000$, while benchmarking at large N can be prohibitively expensive. This makes the choice of the evaluation criteria especially important.

Our system operated across several individual workspaces, producing 414 distinct solver designs evaluated by 42 co-evolving objectives. Roughly the first 200 designs came from the top performers and their full genealogies in separate exploratory workspaces; these were then merged into a single workspace where roughly 200 more designs were generated (see SI Sec. IIIC for details). The merging produced the sub-tree structures in Fig. 3(a), with independent lineages eventually converging toward the best design. Over the course of the search, objectives grew steadily more sophisticated, from simple power-law fits early on to schedule-faithful reach metrics that emphasize the largest solvable problem size, penalize

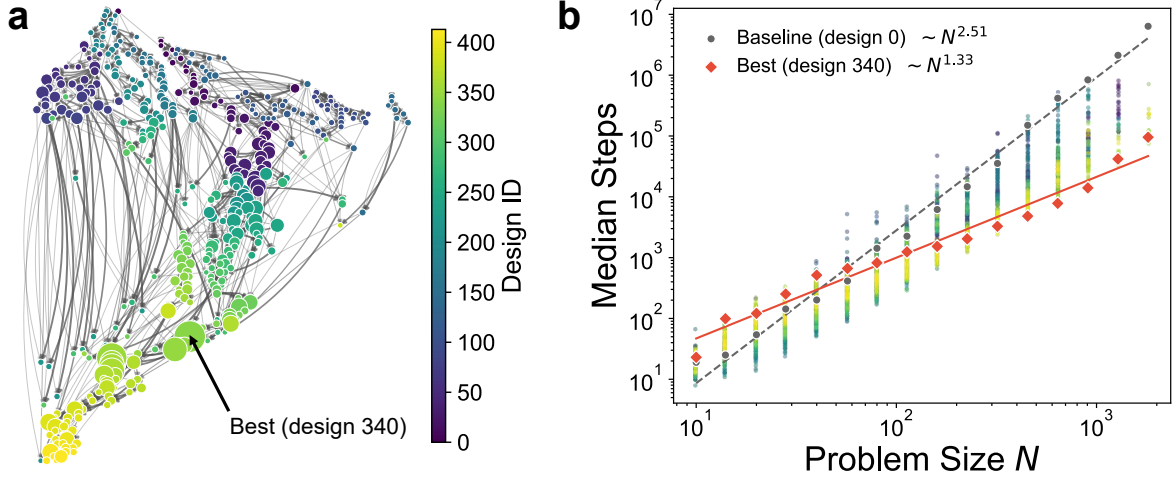


FIG. 3. **Algorithm discovery for 3-SAT DMM solvers.** (a) Design genealogy graph. Nodes are solver designs colored by ID (chronological); larger nodes rank higher under the consensus. Directed edges represent the reference weights. Sub-tree structures arise from merging several exploratory workspaces, with diverse lineages converging toward the best design 340. (b) Scaling of median solution steps with problem size N . The baseline DMM solver scales as $N^{2.51}$; the best solver (design 340) scales as $N^{1.33}$ —a $\sim 67\times$ speedup at $N = 1810$ (95,503 vs. 6,369,516 median steps). Background points cover all 414 designs, colored by ID to show progressive improvement across the search. Note that earlier designs frequently perform better on smaller N , but later designs achieve much stronger performance on larger N .

budget cliffs, and analyze tail scaling.

Fig. 3(b) plots the scaling of median steps-to-solution vs. the problem size N for all designs. The baseline DMM solver [32] scales as $N^{2.51}$ and requires 6,369,516 median steps to solve random 3-SAT instances at $N = 1810$ and clause-to-variable ratio 4.3. Design 340, the best solver found, scales as $N^{1.33}$ and brings the median steps down to 95,503 at $N = 1810$ —a $\sim 67\times$ speedup. All 414 designs are plotted in the background, colored by design ID. Earlier solvers often perform better at smaller N , but their performance degrades more steeply with increasing problem size. In SI Sec. I we provide a detailed analysis of the best solver’s architecture, genealogy, and scaling behavior.

The multi-fidelity schedule was essential for efficiency. Every design was first evaluated at low fidelity to filter out poor candidates quickly. Fidelity promotion was rule-based: the top 50% of designs (by consensus ranking) advance to medium fidelity, then the top 10% to high fidelity. We also tried an LLM-based judge for promotion decisions but found the simpler rules more reliable (SI Sec. III B).

III. DISCUSSION

We have demonstrated that treating scientific discovery as meta-optimization by co-evolving solutions and evaluation criteria can produce significant improvements over fixed-objective optimization. Consensus objective aggregation offers a rigorous solution to the objective specification and the reward hacking problem faced by

automated research systems, and the objective portfolio adapts as the research matures.

This consensus mechanism plays a key role in our 3-SAT DMM case study. Early objectives, built around power-law extrapolation, provided useful initial direction but were gradually replaced by metrics that assessed scaling at larger problem sizes (see SI Sec. II). As new objectives entered and old ones lost weight through age decay, the consensus ranking evolved gradually, making the research process stable and consistent.

Still, several limitations are worth addressing. Because all objectives come from the same LLM backbone, they tend to share assumptions, and agreement-based weighting can reinforce those shared biases instead of filtering noise, causing an “echo chamber” effect. During research, we observed early objectives converged on similar power-law extrapolation metrics until the meta-agent stepped in to suppress redundant ones and steer later generations toward greater diversity. The meta-agent’s weight multipliers offer a top-down correction to the agreement signal (Sec. II B), but more principled diversity-promoting mechanisms would further strengthen the guard against collective blind spots.

There is also an inherent tension in how much autonomy the system should have. We opted for a structured design: fixed agent roles, predefined workflow stages, and guardrails constraining what each agent can touch. The result is predictable and safe, though less flexible than agentic systems with full authorization in a sandbox environment. Our preliminary experiments with fewer restrictions revealed a few failure modes, including emergent goal drift, “cheating” by modifying evalu-

ation scripts, and cherry-picking favorable results. How to balance the safety of structured workflows against the creative reach of unconstrained agents remains an important challenge.

IV. METHODS

A. Agent Architecture

All four agents run on GPT 5.2 [41], with structured outputs enforced through Pydantic schemas for reliable parsing. The planner takes in the MCGS-ranked design list together with a summary of the full experiment history (genealogies, performance metrics, failure patterns) and returns N strategic research directions, each specifying reference design IDs and a natural-language description of the intended modification. The designer receives one direction, writes Python solver code, and launches multi-fidelity experiments. Every 20 designs, the meta-agent computes Kendall’s τ correlations between all objectives, assigns weight multipliers, identifies the current research phase, and issues strategic guidance to the objective agent for subsequent rounds. The objective agent then produces a Python function, mapping experiment results to a scalar score. A HEBO [35] hyperparameter optimization module runs 50 iterations on the best untuned design whenever the tuned fraction drops below 5%, refining continuous hyperparameters without changing the solver architecture. The methodology evolution of the framework is documented in the SI Sec. III, and the modularized solver framework is documented in the SI Sec. IV. All agent conversations are logged for reproducibility, and complete prompt templates can be found in the SI Sec. V.

B. Consensus Objective Algorithm

Given K objective functions $\{f_1, \dots, f_K\}$ and n designs $\{d_1, \dots, d_n\}$ with experiment results, the consensus objective is computed as follows:

1. **Score matrix:** $S_{ij} = f_i(d_j)$ for each objective f_i and design d_j . Invalid or missing scores are set to $+\infty$.
2. **Rank matrix:** For each objective i , designs are sorted by S_{ij} and assigned integer ranks $R_{ij} \in \{0, 1, \dots, n-1\}$.
3. **Kendall’s τ matrix:** For each pair (i, k) , compute $\tau_{ik} = \text{KendallTau}(\{R_{ij}\}_j, \{R_{kj}\}_j)$.
4. **Objective weights:**

$$w_i = \frac{\max(\tilde{\tau}_i, 0) \cdot \lambda^{t-t_i}}{\sum_{k=1}^K \max(\tilde{\tau}_k, 0) \cdot \lambda^{t-t_k}}, \quad (2)$$

where $\tilde{\tau}_i = \text{median}_{k \neq i} \tau_{ik}$ is the median pairwise correlation, $\lambda = 0.9$ is the age decay base, t is the current round, and t_i is the creation round of objective i .

5. **Meta-agent adjustment (optional):** If meta-agent multipliers $\{m_i\}$ are provided, weights are updated as $w'_i = w_i \cdot m_i / \sum_k w_k \cdot m_k$.

6. **Consensus score:**

$$C_j = \sum_{i=1}^K w_i \cdot \frac{R_{ij}}{n-1}, \quad (3)$$

where $C_j \in [0, 1]$ with lower values indicating better designs. This is a weighted Borda count [28] that aggregates normalized ranks across all weighted objectives.

For designs evaluated on-the-fly during the search and not yet stored in the database, normalized ranks are estimated by binary search into each objective’s sorted score distribution.

C. Monte Carlo Graph Search

We adapt Monte Carlo Graph Search (MCGS) [38] from board games to the design space. Each design is a node in a directed acyclic graph whose edges are defined by the reference weights in the design’s genealogy: when the designer creates design d_j from parents $p_1, \dots, p_m$ with weights $\omega_1, \dots, \omega_m$ (self-reported, normalized to sum to 1), edges $(p_k \rightarrow d_j)$ are added for every $k \in [1, \dots, m]$.

Visit counts n_j for each design d_j propagate upward by breadth-first search. Creating design d_j with reference weight ω_k to parent p_k adds ω_k to p_k ’s visit count n_k . Propagation continues recursively, each hop from depth d to $d+1$ attenuated by the ancestor’s edge weight times κ^{d+1} ($\kappa = 0.9$); deeper ancestors therefore receive rapidly diminishing credit. Propagation halts once contributions drop below 10^{-4} .

The UCB score (Eq. (1)) combines the exploitation term r_j (normalized consensus rank; the best design scores 1.0) and the exploration term ($c \cdot \frac{\sqrt{N_{\text{total}}}}{1+n_j}$), with $N_{\text{total}} = \sum_j n_j$ and $c = 0.1$ setting the balance. Rebuilding the graph each iteration from the current consensus objective keeps rankings updated with the latest evaluation criteria.

D. Multi-Fidelity Schedule

Computational resources are allocated progressively across three fidelity tiers. Problem sizes follow a geometric progression $N_k = \text{round}(N_0 \cdot 2^{k/2})$ with $N_0 = 10$ and $k = 0, 1, 2, \dots$, yielding the sequence

$N \in \{10, 14, 20, 28, 40, \dots\}$. The solver step budget (`max_steps`) starts at 50 and doubles at each level.

- **Low fidelity:** $N \leq 640$, `max_steps` up to 10^4 , timeout after 1 minute. Fast initial screening.
- **Medium fidelity:** $N \leq 1280$, `max_steps` up to 10^5 , timeout after 5 minutes. Moderate evaluation for promising candidates.
- **High fidelity:** $N \leq 2560$, `max_steps` up to 10^6 , timeout after 30 minutes. Thorough benchmark of the best designs.

At every level, the solver is run on 100 random planted 3-SAT instances [29] per problem size at clause-to-variable ratio $\alpha_r = 4.3$ (near the complexity peak [42]). A problem size is considered solved if at least half of the instances finish within the step budget. Fidelity promotion is rule-based: designs in the top 50% by consensus advance to medium fidelity; those in the top 10% further advance to high fidelity. The scheme allocates computational effort toward the strongest candidates and filters out weak designs early.

E. Baseline 3-SAT DMM Algorithm

A random 3-SAT instance has N Boolean variables $V_i \in \{0, 1\}$ and $M = \lfloor \alpha_r N \rfloor$ clauses, each being the disjunction of three literals. The baseline DMM algorithm relaxes every variable to a continuous value $v_i \in [-1, 1]$ and introduces auxiliary memory variables: a long-term clause penalty weight $x_{l,m} \in [1, 10^6]$ that grows for persistently violated clauses, and a short-term satisfaction switch $x_{s,m} \in [0, 1]$ toggling between push and hold modes [32]. The coupled ordinary differential equations governing the dynamics are:

$$\dot{v}_n = \sum_{m=1}^M x_{l,m} x_{s,m} G_{n,m} + (1 + \zeta x_{l,m})(1 - x_{s,m}) R_{n,m}, \quad (4)$$

$$\dot{x}_{s,m} = \beta (x_{s,m} + \epsilon) (c_m - \gamma), \quad (5)$$

$$\dot{x}_{l,m} = \alpha (c_m - \delta), \quad (6)$$

The literal polarity $q_{n,m} = \pm 1$ indicates whether variable n appears positive or negated in clause m . The clause satisfaction monitor,

$$c_m(v_i, v_j, v_k) = \frac{1}{2} \min[(1 - q_{i,m} v_i), (1 - q_{j,m} v_j), (1 - q_{k,m} v_k)], \quad (7)$$

ranges between 0 (satisfied) and 1 (violated). $\alpha, \beta, \gamma, \delta, \epsilon, \zeta$, and the integration step size Δt_0 are hyperparameters [32]. The gradient term

$G_{n,m} = \frac{1}{2} q_{n,m} \min[(1 - q_{j,m} v_j), (1 - q_{k,m} v_k)]$, (8) pushes variable v_n toward its satisfying polarity, scaled by how close the other two literals j, k in the clause are to being satisfied. The rigidity term

$$R_{n,m} = \begin{cases} \frac{1}{2}(q_{n,m} - v_n), & \text{if } c_m = \frac{1}{2}(1 - q_{n,m} v_n), \\ 0, & \text{otherwise,} \end{cases} \quad (9)$$

fires only when variable n is the most satisfied literal in clause m , pulling it back toward its satisfying value to keep the clause from flipping.

Finally, the numerical integration time step Δt is rescaled by the inverse of the maximum absolute v -derivative:

$$\Delta t = \frac{1}{\max_n |\dot{v}_n|} \Delta t_0, \quad (10)$$

with the denominator lower-bounded at 10^{-6} for numerical stability. In practice, $\max_n |\dot{v}_n|$ will not reach 0 unless the system converges exactly at the solution point (which is a property of DMMs [30, 32]), and this rescaling constitutes an adaptive time step, stabilizing the numerical integration.

Author Contributions— Y.-H.Z. suggested and M.D. supervised the work. Y.-H.Z. designed and performed the numerical simulations. All authors contributed to the scientific discussion, and have read and approved the final manuscript.

Data availability— The database of all designs with experiment results and all objective functions is available at [43].

Code availability— The complete framework code, including all agent prompts, the consensus objective algorithm, Monte Carlo Graph Search implementation, and the 3-SAT DMM solver, is available at [43].

Acknowledgments. This work was funded by the National Science Foundation via grant No. ECCS-2229880. M.D. also acknowledges funding by the Alexander von Humboldt Stiftung through the Humboldt Research Award.

-
- [1] M. Di Ventra, *The Scientific Method: Reflections from a Practitioner* (Oxford University Press, Oxford, 2018).
 - [2] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, [The AI Scientist: Towards Fully](#)

[Automated Open-Ended Scientific Discovery](#) (2024), [arXiv:2408.06292 \[cs\]](#).

- [3] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha, [The AI Scientist-v2: Workshop-](#)

- Level Automated Scientific Discovery via Agentic Tree Search (2025), [arXiv:2504.08066 \[cs\]](#).
- [4] J. Gottweis, W.-H. Weng, A. Daryin, T. Tu, A. Palepu, P. Sirkovic, A. Myaskovsky, F. Weissenberger, K. Rong, R. Tanno, K. Saab, D. Popovici, J. Blum, F. Zhang, K. Chou, A. Hassidim, B. Gokturk, A. Vahdat, P. Kohli, Y. Matias, A. Carroll, K. Kulkarni, N. Tomasev, Y. Guan, V. Dhillon, E. D. Vaishnav, B. Lee, T. R. D. Costa, J. R. Penadés, G. Peltz, Y. Xu, A. Pawlosky, A. Karthikesalingam, and V. Natarajan, [Towards an AI co-scientist](#) (2025), [arXiv:2502.18864 \[cs\]](#).
 - [5] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, Autonomous chemical research with large language models, *Nature* **624**, 570 (2023).
 - [6] L. Mitchener, A. Yiu, B. Chang, M. Bourdenx, T. Nadolski, A. Sulovari, E. C. Landsness, D. L. Barabasi, S. Narayanan, N. Evans, S. Reddy, M. Foiani, A. Kamal, L. P. Shriver, F. Cao, A. T. Wassie, J. M. Laurent, E. Melville-Green, M. Caldas, A. Bou, K. F. Roberts, S. Zagorac, T. C. Orr, M. E. Orr, K. J. Zvezdaryk, A. E. Ghareeb, L. McCoy, B. Gomes, E. A. Ashley, K. E. Duff, T. Buonassisi, T. Rainforth, R. J. Bateman, M. Skarliniski, S. G. Rodrigues, M. M. Hinks, and A. D. White, [Kosmos: An AI Scientist for Autonomous Discovery](#) (2025), [arXiv:2511.02824 \[cs\]](#).
 - [7] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, Mathematical discoveries from program search with large language models, *Nature* **625**, 468 (2024).
 - [8] A. Novikov, N. Vü, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog, [AlphaEvolve: A coding agent for scientific and algorithmic discovery](#) (2025), [arXiv:2506.13131 \[cs\]](#).
 - [9] C. Yu, R. Liang, C.-T. Ho, and H. Ren, [Autonomous Code Evolution Meets NP-Completeness](#) (2025), [arXiv:2509.07367 \[cs\]](#).
 - [10] A. Merchant, S. Batzner, S. S. Schoenholz, M. Aykol, G. Cheon, and E. D. Cubuk, Scaling deep learning for materials discovery, *Nature* **624**, 80 (2023).
 - [11] F. Y. Wang, D. S. Lee, D. L. Kaplan, and M. J. Buehler, [Swarms of Large Language Model Agents for Protein Sequence Design with Experimental Validation](#) (2025), [arXiv:2511.22311 \[cs\]](#).
 - [12] X.-Q. Han, Z.-F. Gao, P.-J. Guo, and Z.-Y. Lu, [PhysAgent: A Multi-Agent Approach to the Automated Discovery of Physical Laws](#) (2025).
 - [13] Z. Zheng, Z. Xie, Z. Wang, and B. Hooi, [Monte Carlo Tree Search for Comprehensive Exploration in LLM-Based Automatic Heuristic Design](#) (2025), [arXiv:2501.08603 \[cs\]](#).
 - [14] H. Wang, X. Zhang, and C. Mu, [Planning of Heuristics: Strategic Planning on Large Language Models with Monte Carlo Tree Search for Automating Heuristic Optimization](#) (2025), [arXiv:2502.11422 \[cs\]](#).
 - [15] H. Wang and L. Zeng, [Automated Algorithmic Discovery for Scientific Computing through LLM-Guided Evolutionary Search: A Case Study in Gravitational-Wave Detection](#) (2025), [arXiv:2508.03661 \[cs\]](#).
 - [16] D. Zhang, X. Huang, D. Zhou, Y. Li, and W. Ouyang, [Accessing GPT-4 level Mathematical Olympiad Solutions via Monte Carlo Tree Self-refine with LLaMa-3 8B](#) (2024), [arXiv:2406.07394 \[cs\]](#).
 - [17] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, [Tree of Thoughts: Deliberate Problem Solving with Large Language Models](#) (2023), [arXiv:2305.10601 \[cs\]](#).
 - [18] J. Wei, Y. Yang, X. Zhang, Y. Chen, X. Zhuang, Z. Gao, D. Zhou, G. Wang, Z. Gao, J. Cao, Z. Qiu, M. Hu, C. Ma, S. Tang, J. He, C. Song, X. He, Q. Zhang, C. You, S. Zheng, N. Ding, W. Ouyang, N. Dong, Y. Cheng, S. Sun, L. Bai, and B. Zhou, [From AI for Science to Agentic Science: A Survey on Autonomous Scientific Discovery](#) (2025), [arXiv:2508.14111 \[cs\]](#).
 - [19] T. Zheng, Z. Deng, H. T. Tsang, W. Wang, J. Bai, Z. Wang, and Y. Song, [From Automation to Autonomy: A Survey on Large Language Models in Scientific Discovery](#) (2025), [arXiv:2505.13259 \[cs\]](#).
 - [20] Z. Luo, Z. Yang, Z. Xu, W. Yang, and X. Du, [LLM4SR: A Survey on Large Language Models for Scientific Research](#) (2025), [arXiv:2501.04306 \[cs\]](#).
 - [21] S. Eger, Y. Cao, J. D'Souza, A. Geiger, C. Greisinger, S. Gross, Y. Hou, B. Krenn, A. Lauscher, Y. Li, C. Lin, N. S. Moosavi, W. Zhao, and T. Miller, [Transforming Science with Large Language Models: A Survey on AI-assisted Scientific Discovery, Experimentation, Content Generation, and Evaluation](#) (2025), [arXiv:2502.05151 \[cs\]](#).
 - [22] C. A. E. Goodhart, Problems of Monetary Management: The UK Experience, in *Monetary Theory and Practice: The UK Experience*, edited by C. A. E. Goodhart (Macmillan Education UK, London, 1984) pp. 91–121.
 - [23] J. Skalse, N. Howe, D. Krashennnikov, and D. Krueger, Defining and Characterizing Reward Gaming, *Advances in Neural Information Processing Systems* **35**, 9460 (2022).
 - [24] H. Su, Y. Sun, and C. Yu, [The End of Reward Engineering: How LLMs Are Redefining Multi-Agent Coordination](#) (2026), [arXiv:2601.08237 \[cs\]](#).
 - [25] Y. Du, B. Yu, T. Liu, T. Shen, J. Chen, J. G. Rittig, K. Sun, Y. Zhang, Z. Song, B. Zhou, C. Masschelein, Y. Wang, H. Wang, H. Jia, C. Zhang, H. Zhao, M. Ester, T. Head-Gordon, C. P. Gomes, H. Sun, C. Duan, P. Schwaller, and W. Jin, [Accelerating Scientific Discovery with Autonomous Goal-evolving Agents](#) (2025), [arXiv:2512.21782 \[cs\]](#).
 - [26] R. Lu, Z. Shao, Y. Ding, R. Chen, D. Wu, H. Su, T. Yang, F. Zhang, J. Wang, Y. Shi, Z.-P. Jiang, H. Ding, and H.-T. Zhang, Discovery of the reward function for embodied reinforcement learning agents, *Nature Communications* **16**, 11064 (2025).
 - [27] M. G. KENDALL, A NEW MEASURE OF RANK CORRELATION, *Biometrika* **30**, 81 (1938).
 - [28] P. Emerson, The original Borda count and partial voting, *Social Choice and Welfare* **40**, 353 (2013).
 - [29] W. Barthel, A. K. Hartmann, M. Leone, F. Ricci-Tersenghi, M. Weigt, and R. Zecchina, Hiding Solutions in Random Satisfiability Problems: A Statistical Mechanics Approach, *Physical Review Letters* **88**, 188701 (2002).
 - [30] M. Di Ventra, *MemComputing: Fundamentals and Applications* (Oxford University Press, 2022).
 - [31] F. L. Traversa and M. Di Ventra, Polynomial-time solution of prime factorization and NP-complete problems with digital memcomputing machines, *Chaos: An In-*

- terdisciplinary Journal of Nonlinear Science **27**, 023107 (2017).
- [32] S. R. B. Bearden, Y. R. Pei, and M. Di Ventra, Efficient solution of Boolean satisfiability problems with digital memcomputing, *Scientific Reports* **10**, 19741 (2020).
 - [33] C. Sipling, Y.-H. Zhang, and M. Di Ventra, Phase-space engineering and collective dynamics in memcomputing, *Physical Review Applied* **25**, 014048 (2026).
 - [34] B. Peherstorfer, K. Willcox, and M. Gunzburger, Survey of Multifidelity Methods in Uncertainty Propagation, Inference, and Optimization, *SIAM Review* **60**, 550 (2018).
 - [35] A. Cowen-Rivers, W. Lyu, R. Tutunov, Z. Wang, A. Grosnit, R.-R. Griffiths, A. Maravel, J. Hao, J. Wang, J. Peters, and H. Bou Ammar, Hebo: Pushing the limits of sample-efficient hyperparameter optimisation, *Journal of Artificial Intelligence Research* **74** (2022).
 - [36] O. Berger-Tal, J. Nathan, E. Meron, and D. Saltz, The Exploration-Exploitation Dilemma: A Multidisciplinary Framework, *PLOS ONE* **9**, e95693 (2014).
 - [37] P. Auer, N. Cesa-Bianchi, and P. Fischer, Finite-time Analysis of the Multiarmed Bandit Problem, *Machine Learning* **47**, 235 (2002).
 - [38] J. Czech, P. Korus, and K. Kersting, Improving AlphaZero Using Monte-Carlo Graph Search, *Proceedings of the International Conference on Automated Planning and Scheduling* **31**, 103 (2021).
 - [39] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, Mastering the game of Go with deep neural networks and tree search, *Nature* **529**, 484 (2016).
 - [40] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. Van Den Driessche, T. Graepel, and D. Hassabis, Mastering the game of Go without human knowledge, *Nature* **550**, 354 (2017).
 - [41] OpenAI, Introducing GPT-5.2, <https://openai.com/index/introducing-gpt-5-2/> (2025).
 - [42] A. K. Hartmann and H. Rieger, eds., *New Optimization Algorithms in Physics* (Wiley-VCH ; John Wiley, Weinheim : Chichester, 2004).
 - [43] Y.-H. Zhang, *Github repository: Scientific research as meta-optimization*, Available at: https://github.com/yuanhangzhang98/LLM_meta_optimization (2026).

Supplementary Files

This is a list of supplementary files associated with this preprint. Click to download.

- [supplementarymisubmission.pdf](#)